

Tutorial: Build a “Student Project Showcase” Web App with Cursor

1. Tutorial Goal

Students will use Cursor to build a creative web application called:

Student Project Showcase

This app helps students present their AI, robotics, data science, or creative coding project in a professional way.

By the end, students will create a web app that includes:

A project title, short description, project category, skills used, progress status, project image placeholder, and a “View Project Plan” button.

The main purpose is not advanced programming. The main purpose is to show students how to use Cursor to improve an app from a basic layout into a professional-looking web application using the design principles from Lesson 5.

2. What Students Will Learn

Students will learn how to:

Use Cursor to generate a simple web app.

Identify beginner UI problems.

Ask Cursor for specific design improvements.

Apply spacing, alignment, visual hierarchy, colors, typography, cards, and buttons.

Test whether the app still works after AI changes.

Explain before-and-after design improvements.

This directly matches the Lesson 5 flow: diagnose design problems, use a specific AI prompt, apply styling, test the app, and submit before/after screenshots.

3. App Idea: Student Project Showcase

The app will look like a mini portfolio page.

Example student project:

AI Recycling Helper A web app that helps users identify recyclable items and learn how to sort waste correctly.

The final app should have:

Project title Subtitle Project summary Technology tags Progress cards Main action button Clean dashboard-style design Responsive layout

This is a good middle/high school project because students can easily customize it with their own project ideas: robotics, AI music, game design, environmental science, sports analytics, photography AI, etc.

4. Before Class Setup

Students need:

Cursor installed Node.js installed Basic HTML/CSS/JavaScript knowledge A folder for the project

Recommended project stack:

Vite + React

This is easier than full Next.js for beginners and works well for a one-lesson web app.

5. Step-by-Step Student Tutorial

Step 1: Create a New React Project

Open Cursor.

Open a terminal inside Cursor.

Run:

```
npm create vite@latest student-project-showcase -- --template react
cd student-project-showcase
npm install
npm run dev
```

Then open the local URL shown in the terminal, usually:

```
http://localhost:5173
```

Students should see the default Vite React page.

6. Step 2: Ask Cursor to Create a Basic App First

Open `src/App.jsx`.

Select all existing code and ask Cursor:

```
Create a simple React web app called Student Project Showcase.
```

```
The app should show:
```

- A project title
- A short project description
- A project category
- A list of skills used
- A progress section with 3 milestones
- A button labeled "View Project Plan"

```
Keep the code simple for middle and high school students.
```

```
Use only React and CSS.
```

```
Do not add external libraries.
```

At this point, the app may work, but it may still look basic. That is useful because students need a "before" version.

Ask students to take a screenshot and name it:

```
before-design.png
```

7. Step 3: Diagnose Beginner Design Problems

Ask students to look at their app and write down 3 problems.

Use the Lesson 5 checklist:

Does the app have enough space? Are the sections aligned? Is the most important information obvious? Are the colors readable? Is the text easy to scan? Do the cards look professional? Does the button clearly tell the user what to do?

This connects to the lesson's "Diagnosing Common Beginner Problems" section, which points out problems like too much text, random colors, tiny buttons, no obvious main action, and messy fonts.

Example student answers:

The project description is too close to the title. The button is small and not easy to notice. The milestones are not organized into cards.

8. Step 4: Use a Professional UI Prompt in Cursor

Now students should not say:

```
Make it pretty.
```

Instead, they should use a specific design prompt.

Paste this into Cursor:

```
Improve the UI design of this app using a clean, modern dashboard style.

Target user:
Middle and high school students presenting a creative project.

Design requirements:
- Use a centered page layout with a maximum width.
- Put the main project information inside a large hero card.
- Use smaller cards for the project milestones.
- Add generous spacing and padding so the app has room to breathe.
- Use clear visual hierarchy: large title, smaller subtitle, readable body text.
- Use one bright primary color for the main button.
- Use a simple 2-4 color palette.
- Make the button large, rounded, and action-oriented.
- Keep the text high contrast and easy to read.
- Use consistent alignment.
- Keep the code simple and beginner-friendly.

Golden rule:
Do NOT change the app's features. Only improve the layout and visual design.
```

This prompt follows the lesson's "Prompt Like an Art Director" idea: target user, design style, layout, colors/fonts, and the golden rule: do not change the app's features.

9. Step 5: Improve the App Content

Now students can make the app more creative.

Ask Cursor:

```
Update the sample project content to make it more interesting.

Use this project idea:
AI Recycling Helper
```

Description:

A student-built web app that helps people learn whether an item belongs in recycling, compost, trash, or special disposal.

Skills:

React, JavaScript, UI Design, Environmental Science, AI Prompting

Milestones:

1. Build the recycling category database
2. Design the user search interface
3. Add AI-powered sorting suggestions

Keep the code simple.

Students can replace the project idea with their own.

Other possible project ideas:

AI Homework Planner Smart Traffic Light Simulator K-pop Dance Practice Tracker Robotics Competition Dashboard AI Photography Coach Personal Finance Game for Teens Mental Math Battle App Climate Data Explorer

10. Step 6: Add One Interactive Feature

Now ask Cursor to add a simple feature without making the app too complex:

Add a simple interactive feature.

When the user clicks "View Project Plan", show or hide a project plan section below the button.

The project plan section should include:

- Week 1: Research the problem
- Week 2: Build the first prototype
- Week 3: Test with users
- Week 4: Improve the design and prepare a demo

Keep the implementation simple using React useState.
Do not add any external libraries.

This teaches students that good design should not break functionality.

After Cursor edits the app, students should test:

Does the button work? Does the project plan appear? Does it hide again when clicked? Is the layout still clean?

11. Step 7: Ask Cursor to Explain the Code

Students should not just accept AI-generated code. They should understand it.

Ask Cursor:

```
Explain this React app to a beginner.
```

```
Please explain:
```

- What App.jsx does
 - What useState does
 - How the button shows and hides the project plan
 - How the CSS creates spacing, cards, colors, and visual hierarchy
- ```
Use simple language for middle and high school students.
```

This helps students connect code with design decisions.

---

## 12. Step 8: Final UI Improvement Prompt

---

Use this final polish prompt:

```
Review the app like a UI designer.
```

```
Check for:
```

- Spacing
- Alignment
- Visual hierarchy
- Color contrast
- Typography
- Card layout
- Button clarity
- Mobile responsiveness

```
Suggest 5 improvements first.
```

```
Then apply the improvements that keep the code simple.
```

```
Do not change the app's core features.
```

This reinforces the lesson's message that professional design comes from specific improvements, not vague "make it better" instructions.

---

## 13. Suggested Final Code Structure

---

Students should have files like:

```
student-project-showcase/
 src/
 App.jsx
 App.css
 main.jsx
 package.json
```

Encourage students to keep the app simple. For Lesson 5, the learning goal is **professional design transformation**, not building a large application.

---

## 14. Example Final App Requirements

---

The final app should include:

A clean page background  
A centered main container  
A large hero card  
A clear project title  
A short subtitle  
Readable body text  
Skill tags  
Three milestone cards  
One primary button  
A show/hide project plan section  
Consistent spacing  
Consistent rounded cards  
High-contrast text  
Mobile-friendly layout

---

## 15. Student Submission Checklist

---

Students should submit:

Before screenshot  
After screenshot  
Their project folder or GitHub link  
Three design problems they fixed  
Two sentences explaining what changed and why

Example reflection:

```
I improved the app by adding more spacing, using cards, and making the main button
easier to see. These changes make the app easier to understand because the most
important information appears first and the project milestones are organized
clearly.
```

This matches the lesson's milestone submission: apply UI improvements, make at least three noticeable visual changes, take before/after screenshots, and explain the changes.

---

## 16. Teacher Demo Script

---

You can teach it like this:

"Today we are not just coding. We are learning how to make our app look professional."

"First, we will ask Cursor to create a basic app."

“Then we will diagnose design problems like real UI designers.”

“After that, we will not say ‘make it pretty.’ We will give Cursor specific design instructions.”

“Finally, we will test the app and explain our design decisions.”

## 17. In-Class Activity

### Think-Pair-Share

Ask students to open their basic app.

**Think:** Find 3 design problems silently. **Pair:** Compare with a partner. **Share:** Tell the class one thing you would improve.

This follows the lesson’s Think-Pair-Share activity on identifying design problems before asking AI to fix them.

## 18. Assessment Rubric

| Category         | Excellent                                                  | Good                       | Needs Improvement               |
|------------------|------------------------------------------------------------|----------------------------|---------------------------------|
| Functionality    | App runs and button works                                  | App runs with minor issues | App does not run                |
| Spacing          | Layout has clear breathing room                            | Some spacing is improved   | Content feels crowded           |
| Alignment        | Sections are consistently aligned                          | Mostly aligned             | Random placement                |
| Visual Hierarchy | Title, subtitle, cards, and button are clearly prioritized | Some hierarchy             | Hard to know what matters       |
| Color            | Uses simple, readable color palette                        | Mostly readable            | Too many colors or low contrast |
| Button Design    | Button is clear and action-oriented                        | Button is visible          | Button is unclear or too small  |
| Reflection       | Clearly explains 3 changes                                 | Explains 1–2 changes       | Little or no explanation        |

## 19. Optional Challenge

Advanced students can ask Cursor:



Add a second project card so this app becomes a mini portfolio with two student projects.

Keep the layout clean and responsive.  
Use the same card style for both projects.  
Do not add external libraries.

Or:

Add a simple category filter with buttons:  
All, AI, Robotics, Data Science, Web App.

When a user clicks a category, only show matching projects.  
Keep the code beginner-friendly.

---

## 20. Key Teaching Message

---

The most important lesson is:

**AI can help you design faster, but you must give clear design direction.**

Students should remember this formula:

Target User + Design Style + Layout + Colors/Fonts + Specific Components + Do Not Change Features

That is the difference between a weak prompt:

Make it pretty.

and a strong prompt:

Use a clean dashboard style, centered cards, generous spacing, high-contrast text, one blue primary button, and do not change the app's features.